



UFOP

Universidade Federal
de Ouro Preto

**Universidade Federal de Ouro Preto
Instituto de Ciências Exatas e Aplicadas
Departamento de Computação e Sistemas**

Um Simulador de maquinas de estados

Matheus Henrique Vieira Ribeiro

**TRABALHO DE
CONCLUSÃO DE CURSO**

**ORIENTAÇÃO:
Elton Máximo Cardoso**

**Julho, 2026
João Monlevade—MG**

Matheus Henrique Vieira Ribeiro

Um Simulador de maquinas de estados

Orientador: Elton Máximo Cardoso

Monografia apresentada ao curso de Sistemas de Informação do Instituto de Ciências Exatas e Aplicadas, da Universidade Federal de Ouro Preto, como requisito parcial para aprovação na Disciplina “Trabalho de Conclusão de Curso II”.

Universidade Federal de Ouro Preto

João Monlevade

Julho de 2026

A Ficha Catalográfica é elaborada exclusivamente pela Biblioteca. Substitua esta página pelo documento gerado na versão final da sua monografia.

A Folha de aprovação deverá ser gerada pelo(a) professor(a) orientador(a) após a realização das correções sugeridas pela banca examinadora.

As instruções para elaboração desse documento eletrônico estão disponíveis em <<https://www.monografias.ufop.br>> (menu “Documentos”, opção “Tutorial Professor Orientador”).

Após gerar a folha de aprovação, o(a) professor orientador(a) enviará o documento ao(à) orientado(a), o qual deverá inseri-lo nesta página.

Este trabalho é dedicado ...

Agradecimentos

Agradeço ...

“Science is more than a body of knowledge; it is a way of thinking.”

— Carl Sagan (1934 – 1996),
in: The Demon-Haunted World: Science as a Candle in the Dark.

Resumo

Segundo a ??, 3.1-3.2), o resumo deve ressaltar o objetivo, o método, os resultados e as conclusões do documento. A ordem e a extensão destes itens dependem do tipo de resumo (informativo ou indicativo) e do tratamento que cada item recebe no documento original. O resumo deve ser precedido da referência do documento, com exceção do resumo inserido no próprio documento. (...) As palavras-chave devem figurar logo abaixo do resumo, antecidas da expressão Palavras-chave:, separadas entre si por ponto e finalizadas também por ponto.

Palavras-chaves: latex. abntex. editoração de texto.

Abstract

This is the english abstract.

Key-words: latex. abntex. text editoration.

Lista de ilustrações

Figura 1 – a Criação do estado, b Editando o estado	34
Figura 2 – a Criação da transição, b Editando a transição	35
Figura 3 – Um APN simples	35
Figura 4 – Entrada da palavra 0101	36
Figura 5 – a Tabela de caracteres, b Visualização dos <i>snapshots</i>	36
Figura 6 – a Caminho de aceitação, b Clique em <i>snapshot</i> , c Botão de avançar . .	37
Figura 7 – a Botões de configuração, b Botões de <i>download/upload</i>	38

Lista de tabelas

Tabela 1 – Trabalhos relacionados	19
---	----

Lista de abreviaturas e siglas

APD Automato de Pilha Determinístico

ER Expressão Regular

LLC Linguagem Livre de Contexto

GLC Gramática Livre de Contexto

Lista de símbolos

ϵ	Letra grega epsilon
α	Letra grega alfa
β	Letra grega beta
γ	Letra grega gamma
λ	Letra grega lambda
$\in$	Pertence

Sumário

1	INTRODUÇÃO	14
1.1	Elaboração do capítulo	14
1.2	O problema de pesquisa	14
1.3	Objetivos	15
1.4	Metodologia	15
1.5	Organização do trabalho	15
2	REVISÃO BIBLIOGRÁFICA	17
2.1	Trabalhos correlatos	17
2.2	Expressões regulares	19
2.3	Linguagens Livres de Contexto	21
2.3.1	Autômatos de Pilha	22
2.3.2	Gramáticas livres de contexto	24
2.4	Derivadas de Expressões Regulares	24
2.5	Derivadas para Linguagens Livres de Contexto	26
3	DESENVOLVIMENTO	30
3.1	Implementação	30
3.1.1	Modelagem do APD e sua execução	31
3.1.2	Gerando palavras a partir de uma GLC	32
3.2	Uso do software	34
4	RESULTADOS	39
5	CONCLUSÃO	40
	REFERÊNCIAS	41

1 Introdução

A teoria da computação estuda as propriedades dos sistemas computacionais. Esta teoria foi formalizada pela primeira vez, de forma efetiva, em 1936, por Alan Turing e Alonzo Church, que construíram as bases do que hoje chamamos de computação, por meio da elaboração do conceito de linguagens formais.

Linguagens formais são lidas por meio de máquinas de estados e escritas por meio de gramáticas formais. Neste trabalho, serão enfatizadas as máquinas de estados, mais especificamente autômatos de pilha não determinísticos (APN), por serem particularmente mais abstratos e de representação mais complexa que as demais máquinas de estados.

Sendo a teoria da computação a base das tecnologias atuais, torna-se essencial que os discentes dos cursos da área de computação, como sistemas de informação e engenharia da computação, dominem esses conceitos para que se tornem bons profissionais. Uma vez que tais conceitos não apenas fornecem a base teórica para a computação, mas também promovem habilidades de pensamento crítico e análise rigorosa, conferindo a capacidade de formalizar problemas, definir algoritmos e analisar sua eficiência.

Como forma de auxiliar os discentes dos cursos da área da computação a visualizar de forma mais clara os autômatos de pilha não determinísticos, este trabalho tem como objetivo desenvolver uma ferramenta web para montagem e visualização de APNs em execução.

1.1 Elaboração do capítulo

Este capítulo apresenta o seu trabalho. Você deve contextualizar o problema abordado, descrever os objetivos gerais e específicos, apresentar a metodologia e como o trabalho está estruturado.

1.2 O problema de pesquisa

Ciente da importância do estudo da teoria da computação, utilizada nos dias de hoje como ferramenta para o desenvolvimento dos compiladores das linguagens de programação modernas, é fundamental que os alunos dos cursos superiores da área de computação dominem esta disciplina. Contudo, eles normalmente precisam lidar com alguns desafios durante o estudo, sendo dois deles:

- O estudo das máquinas de estados é realizado utilizando apenas diagramas em um

papel e abstrair o seu funcionamento dada essa representação limitada pode se tornar um desafio;

- Criar uma maquina de estado que reconheça uma linguagem especifica e apenas ela, é como responder uma charada ou enigma, em que uma resposta precisa satisfazer uma serie de condições especificas para ser considerada correta.

Este trabalho visa amenizar o primeiro desafio e auxiliar no enfrentamento do segundo, facilitando o exercício pratico da disciplina.

1.3 Objetivos

O presente trabalho consiste em facilitar o aprendizado da disciplina de fundamentos teóricos da computação.

Este trabalho possui aos seguintes objetivos específicos:

- Criar um software gráfico que auxilie os alunos simulando o funcionamento de maquinas de estado em tempo real;
- Disponibilizar esse sistema para alunos da disciplina de fundamentos teóricos da computação do campus ICEA;
- Avaliar o impacto do sistema no aprendizado dos alunos por meio de um questionário.

1.4 Metodologia

O objeto de pesquisa deste trabalho ...

Os passos para execução deste trabalho são assim definidos:

- Revisão da literatura
- Desenvolvimento do software de auxilio aos alunos
- Teste por parte dos alunos
- Análise do feedback dos alunos

1.5 Organização do trabalho

É importante observar que a estrutura é apresentada a partir do próximo capítulo. O capítulo de Introdução não deve compor esta descrição. Além disso,

sempre que você fizer referência à algum item específico, a inicial deve ser maiúscula. Por exemplo, Capítulo 2, Tabela 5, Figura 1, dentre outros.

O restante deste trabalho é organizado como se segue. O Capítulo [2](#) apresenta...

2 Revisão bibliográfica

2.1 Trabalhos correlatos

Foram encontrados 10 sistemas com objetivos similares ao proposto por este projeto. Ao longo desta seção, cada um deles será exposto, podendo ser visualizada uma comparação simples dos principais pontos de cada um por meio da tabela 1.

O primeiro foi o Language Emulator ([VIEIRA; VIEIRA; VIEIRA, 2003](#)), projeto desenvolvido em 2003 na UFMG utilizando a linguagem de programação Java pelos pesquisadores Luiz Filipe Menezes Vieira, Marcos Augusto Menezes Vieira e Newton José Vieira. O projeto se trata de um simulador de autômatos que tem foco nas principais operações envolvendo autômatos finitos determinísticos, autômatos finitos não determinísticos com e sem transição lambda, máquinas de Moore e máquinas de Mealy, juntamente com as gramáticas regulares e as expressões regulares. As operações realizadas por esse sistema incluem transformações entre diferentes tipos de máquinas de estados e gramáticas, assim como a capacidade de determinar se uma palavra pertence ou não a essa gramática. Apesar de ser um projeto bastante completo em relação às suas funcionalidades, ele não apresenta uma interface que detalhe com diagramas essas operações, sendo interativa apenas por meio de botões e textos.

O segundo sistema foi AUTOMATOGRAPH ([KIENETZ; CANAL, 2004](#)), desenvolvido por Eduardo Bacchi Kienetz e Ana Paula Canal na UFRGS no ano de 2006 utilizando a linguagem de programação Java. Ele consegue reconhecer palavras de uma linguagem regular pertencentes à linguagem de um autômato informado pelo usuário e gerar uma gramática regular a partir desse autômato; porém, assim como o anterior, não possui uma interface gráfica que represente o diagrama desses autômatos.

Em seguida, o Ginix Abstract Machine (GAM) ([JUKEMURA; NASCIMENTO; UCHÔA, 2005](#)), que é um sistema desenvolvido por Anibal Jukemura, Hugo Nascimento e Joaquim Uchôa na UFLA no ano de 2005. Tem a capacidade de representar em forma de grafos direcionados AFDs, AFNs, AFNLs, APDs, APNs e MTs, além de reconhecer se uma palavra pertence ou não à máquina de estados representada por esse grafo. O GAM também consegue transformar AFNs em AFDs e minimizar AFDs.

O FSM Simulator ([ZUZAK; JANKOVIC, 2025](#)) é um aplicativo web desenvolvido por Ivan Zuzak e Vedrana Janković utilizando HTML, JavaScript e Bootstrap. O FSM Simulator consegue representar o funcionamento e criar aleatoriamente AFDs, AFNs e expressões regulares. Além disso, consegue criar palavras aleatórias e realizar testes. Sua representação se dá por meio de dígrafos estáticos.

AutoSim ou Automaton Simulator ([BURCH, 2008](#)) é um software desenvolvido em Java por Carl Burch na Universidade Carnegie Mellon no ano de 2006. AutoSim é capaz de construir e simular de forma dinâmica AFDs, AFNs, APDs e MTs por meio de grafos direcionados.

Java Finite Automata Simulation Tool ou jFAST ([WHITE; WAY, 2006](#)) é um software que foi desenvolvido na Universidade de Villanova por Timothy M. White e Thomas P. Way no ano de 2006 utilizando a linguagem de programação Java e traz a proposta de construir diagramas dinâmicos para simular o comportamento de AFDs, AFNs, APDs e máquinas de Turing.

Finite State Machine Designer (FSMD) é um aplicativo web desenvolvido por Evan Wallace em 2010 com o objetivo de criar com facilidade diagramas que representassem máquinas de estados em geral e portar esses diagramas para vários formatos como PNG, JSON e LaTeX, porém não simula seu funcionamento, apenas desenha as máquinas.

Automaton Simulator ([DICKERSON, 2021](#)) é um aplicativo web criado por Kyle Dickerson em 2021 e tem o propósito de simular o comportamento de AFDs, AFNs e APDs por meio de diagramas.

UC Davis Automaton Simulator (UCDAS) ([DOTY, 2020](#)) é uma aplicação web criada em 2020 por David Doty utilizando a linguagem de programação Elm na Universidade da Califórnia em Davis e se propõe a ser um simulador que demonstra o comportamento de alguns autômatos e gramáticas, sendo: AFDs, AFNs, Regex, GLCs e MTs. Porém, não possui uma interface com diagramas, sendo sua interatividade realizada por meio de texto e botões.

Por fim, o sistema mais completo é o JFLAP ([RENSSELAER; UNIVERSITY, 2018](#)), que teve seu desenvolvimento iniciado em 1990 na Rensselaer por Dan Caugherty e desde então teve a contribuição de vários desenvolvedores utilizando várias tecnologias como C++, Java e JavaScript, tendo o desenvolvimento transferido para a Duke University em 1995 e sua última atualização até o momento em 2018. Atualmente, o JFLAP consegue criar e simular AFD, AFN, APD, APN e MT, além de transformar cada autômato em sua respectiva gramática e transformar autômatos que são equivalentes entre si, como AFD em AFN e outras operações como minimizar AFDs. Ele realiza todas essas funções com diagramas e elementos gráficos que facilitam bastante o entendimento.

Após visitar estes sistemas, foi possível perceber que apenas o simulador GAM simulava a execução de autômatos de pilha não determinísticos; porém, ele não possuía uma visualização satisfatória do não determinismo desse tipo de autômato, pois cada execução paralela possui sua própria pilha, estado atual e leitura de caractere da palavra, sendo difícil visualizar tantos dados simultaneamente de cada uma das execuções paralelas. Portanto, optou-se por criar um sistema que tivesse como um dos pontos principais representar esse

Sistema	Ano	Linguagem	Idioma	Autômatos	Operações	Interface
Language Emulator	2003	Java	Português	3	Sim	Apenas texto
AUTOMATOCORAPH	2004	Java	Português	2	Não	Apenas texto
GAM	2005	-	Português	6	Sim	Gráfica estática
FSM Simulator	-	JavaScript	Inglês	2	Não	Gráfica estática
AutoSim	2006	Java	Inglês	4	Não	Gráfica dinâmica
jFAST	2006	Java	Inglês	4	Não	Gráfica dinâmica
FSMD	2010	JavaScript	Inglês	1	Não	Gráfica dinâmica
Automaton Simulator	2021	JavaScript	Inglês	3	Não	Gráfica dinâmica
UCDAS	2020	Elm	Inglês	4	Não	Apenas texto
JFLAP	1990-2018	-	Inglês	5	Sim	Gráfica dinâmica

Tabela 1 – Trabalhos relacionados

não determinismo de forma aprimorada.

2.2 Expressões regulares

Esta seção apresenta uma rápida introdução ao conceito de expressões regulares, incluindo a definição, e um sumário das propriedades de linguagens regulares. Como este trabalho se destina principalmente às linguagens livres de contexto, não entraremos em detalhes técnicos e formais a respeito desta matéria. O principal objetivo desta seção é prover ao leitor o conhecimento mínimo necessário para o conceito de derivadas que é apresentado a seguir, já que os autores consideram ser mais natural abordar o tema de derivadas em linguagens regulares antes de se apresentar o conceito sobre derivadas de linguagens livres de contexto.

Dado um alfabeto Σ , uma Expressão Regular ([ER](#)) sobre Σ é definida indutivamente pelo seguinte conjunto de regras:

- $\emptyset$ denota a linguagem vazia.
- A palavra vazia ϵ é uma ER.
- Se $a \in \Sigma$, então a é uma ER.
- Se e_1 e e_2 são expressões regulares, então a concatenação $e_1 e_2$ e a união $e_1 | e_2$ também são expressões regulares.
- Se e é uma expressão regular, então o fecho de Kleene e^* também é uma expressão regular.
- Apenas são expressões regulares aquelas obtidas por uma sequência de aplicações finitas das regras anteriormente descritas.

Sintaticamente, parênteses podem ser empregados para desambiguar o significado das expressões. Vamos considerar como ordem de precedência, da mais alta para a mais baixa, os operadores Kleene, sequência e alternativa. Por exemplo, $ab|cd^*$ é equivalente a $(ab)|c(d^*)$.

A linguagem denotada por uma expressão regular e , dada pela notação $L(e)$, pode ser expressa, por meio de equivalência à teoria de conjuntos, da seguinte forma:

- $L(\emptyset) = \{\}$. A ER para a linguagem vazia denota o conjunto vazio.
- $L(\epsilon) = \{\epsilon\}$, onde ϵ é a palavra vazia. A ER para a palavra vazia denota um conjunto cujo único elemento é a palavra vazia (ou equivalentemente o conjunto unitário vazio).
- $L(a)$, com $a \in \Sigma = \{a\}$. A ER para um símbolo a é um conjunto contendo apenas o referido símbolo.
- $e_1 e_2 = \{w_1 w_2 \mid w_1 \in L(e_1) \wedge w_2 \in L(e_2)\}$. A ER para a concatenação de duas linguagens é o conjunto formado pela justaposição de uma palavra de e_1 seguida por uma palavra de e_2 , nessa ordem.
- $e_1 | e_2 = L(e_1) \cup L(e_2)$. A ER para a alternativa entre duas ERs e_1 e e_2 é dada pela união das palavras em $L(e_1)$ e $L(e_2)$.
- $e^* = \{\epsilon\} \cup L(e) \cup L(e e) \cup L(e e e) \cup \dots$. O fecho de Kleene pode ser entendido como uma união de concatenações da expressão e consigo mesma, de zero até infinitas vezes.

Considere a expressão regular $(a \mid b)^* a$. Podemos facilmente verificar que a palavra aba pertence à linguagem gerada por essa expressão regular. Mas a palavra b não pertence.

$$(a \mid b)^* a$$

$$(a \mid b)(a \mid b)^* a$$

$$(a \mid b)(a \mid b)^* a$$

$$(a \mid b)(a \mid b) \in a$$

$$a(a \mid b) \in a$$

$$ab \in a$$

$$aba$$

Expressões regulares são particularmente úteis em diversos sistemas computacionais como um meio de pesquisa avançada em textos. Por exemplo, o comando `grep` ([Free Software Foundation, 2026](#)) do Linux, que permite listar apenas as linhas de um arquivo onde um padrão dado por uma expressão ocorre.

Como resultados consolidados da teoria da computação, sabe-se que expressões regulares denotam o mesmo conjunto das linguagens regulares, o mesmo reconhecido por autômatos finitos determinísticos (AFD) e autômatos finitos não determinísticos (AFN). Também sabe-se que o conjunto das linguagens regulares é fechado sob uma coleção de operações como união, interseção, complemento, concatenação, reverso, intercalação, entre outras. Esses resultados podem ser facilmente encontrados em livros de teoria, como ([VIEIRA, 2006](#)) e ([HOPCROFT; MOTWANI; ULLMAN, 2006](#)), e não serão replicados neste trabalho.

2.3 Linguagens Livres de Contexto

Linguagens livres de contexto Linguagem Livre de Contexto ([LLC](#)), são uma classe de linguagens formais que podem ser geradas por gramáticas livres de contexto e podem ser reconhecidas por autômatos de pilha. Linguagens livres de contexto são fechadas sob as operações de união, concatenação e fecho de Kleene, ou seja, a aplicação dessas operações resulta em uma linguagem que certamente pertence ao grupo das LLCs. Porém, LLCs não são fechadas sob as operações de interseção e complemento, então não há garantias de que a linguagem resultante dessas operações continue sendo livre de contexto.

2.3.1 Autômatos de Pilha

Um Automato de Pilha Determinístico (APD) é uma máquina de estados formalmente denotada por uma sêxtupla $\langle E, \Sigma, \Gamma, \delta, i, F \rangle$ em que:

- E é um conjunto finito de elementos, ditos estados.
- Σ , também chamado de alfabeto, é um conjunto finito de elementos distintos, ditos símbolos.
- Γ , também chamado de alfabeto da pilha, é um conjunto finito de elementos distintos, ditos símbolos.
- $\delta : E \times \Sigma_\lambda \times \Gamma_\lambda \rightarrow E \times \Gamma_\lambda$

Uma configuração instantânea tem como função demonstrar o estado atual do autômato a cada passo de sua execução. Em um APD, é uma tripla $[e, y, z]$ em que e é o estado atual, y é a palavra que ainda não foi lida e z é a palavra que representa o conteúdo da pilha.

Uma das linguagens que é possível reconhecer com um APD é a linguagem $\{a^n b^n \mid n \geq 0\}$, que pode ser definida formalmente pela seguinte sêxtupla: $(\{1, 2\}, \{a, b\}, \{X\}, \delta, 1, \{1, 2\})$, em que δ é dada por:

$$\delta(1, a, \lambda) = [1, X];$$

$$\delta(1, b, X) = [2, \lambda];$$

$$\delta(2, b, X) = [2, \lambda];$$

Para a palavra $aabb$, a execução do APD é dada por:

$$[1, aabb, \lambda]$$

$$\vdash [1, abb, X]$$

$$\vdash [1, bb, XX]$$

$$\vdash [2, b, X]$$

$$\vdash [2, \lambda, \lambda]$$

A diferença entre um autômato de pilha determinístico e um não determinístico é a presença de transições compatíveis, ou seja, duas ou mais transições que podem ser aplicadas a uma mesma configuração instantânea do autômato, gerando duas configurações

instantâneas diferentes. Isso acontece quando existem duas ou mais transições que possuem o mesmo estado de origem, o mesmo símbolo lido da palavra e o mesmo símbolo no topo da pilha.

Uma das linguagens que é possível reconhecer com um APN é a linguagem $\{w \in \{0,1\}^* \mid n_0(w) = n_1(w)\}$, que pode ser definida formalmente pela seguinte sêxtupla: $(\{=, \neq\}, \{0,1\}, \{Z, U\}, \delta, 1, \{=, =\})$, em que δ é dada por:

$$\delta(=, 0, \lambda) = [\neq, Z];$$

$$\delta(=, 1, \lambda) = [\neq, U];$$

$$\delta(\neq, 0, Z) = [\neq, ZZ];$$

$$\delta(\neq, 0, U) = [\neq, \lambda];$$

$$\delta(\neq, 1, U) = [\neq, UU];$$

$$\delta(\neq, 1, Z) = [\neq, \lambda];$$

$$\delta(\neq, \lambda, \lambda) = [=, \lambda];$$

Para a palavra 0101, a execução do APN é dada por:

$$[=, 0101, \lambda]$$

$$\vdash [\neq, 101, Z]$$

$$\vdash [\neq, 01, \lambda]$$

$$\vdash [\neq, 1, U]$$

$$\vdash [\neq, \lambda, \lambda]$$

A possibilidade de transições compatíveis faz com que APNs sejam máquinas de estados mais poderosas que os APDs, de forma que os APDs não reconhecem todo o conjunto de linguagens livres de contexto, enquanto os APNs reconhecem. Por exemplo, a linguagem $\{a^n b^n c^n \mid n \geq 0\}$ é uma linguagem que não pode ser reconhecida por um APD, mas pode ser reconhecida por um APN. (Obs: nota técnica, a linguagem correta que separa APDs e APNs classicamente é a de palíndromos, pois $a^n b^n c^n$ não é livre de contexto).

2.3.2 Gramáticas livres de contexto

Uma Gramática Livre de Contexto ([GLC](#)) é um conjunto de regras que descrevem a formação de palavras em uma linguagem livre de contexto, em que cada regra da gramática tem no lado esquerdo um único símbolo não terminal e, no lado direito, uma sequência de símbolos terminais e não terminais. As GLCs podem ser representadas formalmente por uma quádrupla (V, Σ, R, S) , em que V é um conjunto finito de símbolos não terminais, Σ é um conjunto finito de símbolos terminais, R é um conjunto finito de regras de produção e S é o símbolo inicial da gramática.

A linguagem livre de contexto $\{0^n 1^n \mid n \geq 0\}$, por exemplo, pode ser gerada pela GLC $G = (\{P\}, \{0, 1\}, R, P)$, em que R é dado por:

$$P \rightarrow 0P1 \mid \lambda$$

O ato de gerar uma palavra a partir de uma GLC é chamado de derivação, que consiste em aplicar as regras da gramática repetidamente partindo do símbolo inicial. Um exemplo de derivação para a palavra 0011 é dado por:

$$\begin{array}{c} P \\ | \\ 0 P 1 \\ | \\ 0 P 1 \\ | \\ \lambda \end{array}$$

2.4 Derivadas de Expressões Regulares

Derivadas de linguagens regulares foram primeiramente descritas por Brzozowski ([BRZOWSKI, 1964](#)) como um meio tanto para determinar a pertinência de uma palavra a um conjunto quanto como forma de deduzir um AFD diretamente a partir de uma expressão regular. Uma derivada de uma linguagem L em relação a um símbolo a pode ser definida como:

$$d(a, L) = \{w \mid aw \in L\} \quad (2.1)$$

Uma derivada em relação a um símbolo a é um conjunto de palavras (e, portanto, uma linguagem) de todos os sufixos w tais que aw faz parte da linguagem original. Antes

de definirmos formalmente a derivada, vamos primeiro apresentar a definição da função $null : e \rightarrow \{T, F\}$ que, dada uma expressão regular e , retorna T se e produz a palavra vazia ou F caso contrário. Chamamos expressões regulares que podem produzir a palavra vazia de anuláveis.

$$\begin{aligned} null(\epsilon) &= T \\ null(a) &= F \\ null(e_1 e_2) &= null(e_1) \wedge null(e_2) \\ null(e_1 \mid e_2) &= null(e_1) \vee null(e_2) \\ null(e^*) &= T \end{aligned}$$

A derivada de uma expressão regular pode ser computada por meio de um conjunto de regras de equivalência que computam uma ER para a linguagem derivada. Considere a expressão que denota a linguagem que contém apenas a palavra vazia ϵ . Pela definição de derivada (2.1), podemos concluir que não existe nenhuma palavra w tal que, para qualquer símbolo do alfabeto, $aw \in \{\epsilon\}$; logo, $d(a, \epsilon) = \emptyset$, para qualquer símbolo $a \in \Sigma$.

O mesmo raciocínio pode ser usado em ER de um único símbolo do alfabeto. Nesse caso, seja $d(a, b)$, em que a e b são símbolos do alfabeto. Se $b = a$, então a derivada deve ser ϵ , pois $b\epsilon \equiv b \in \{b\}$. Por outro lado, se $a \neq b$, então não existe um sufixo w tal que $aw \in \{b\}$.

Um cuidado especial deve ser tomado com a concatenação devido à possibilidade de uma das expressões produzir uma palavra vazia. Ao determinar $d(a, e_1 e_2)$, se e_1 é anulável, devemos considerar a possibilidade de o símbolo a também ser um prefixo de e_2 . Caso contrário, a derivada da sequência será a concatenação $d(a, e_1) e_2$. Como a alternativa representa a união de duas linguagens, a derivada da alternativa é expressa como a alternativa das derivadas das respectivas subexpressões.

Como o fecho de Kleene pode ser visto como uma união de concatenações, a derivada para uma e^* pode ser vista como a derivada para ee^* . A derivada de uma expressão regular em relação a um símbolo a pode ser definida como uma função $d : \Sigma \times e \rightarrow e$. Essa função é definida a seguir:

$$\begin{aligned}
d(a, \epsilon) &= \emptyset \\
d(a, a) &= \epsilon \\
d(a, b) &= \emptyset, \text{ se } a \neq b \\
d(a, e_1 e_2) &= \begin{cases} d(a, e_1) e_2, & \text{se } \neg \text{null}(e_1) \\ d(a, e_1) e_2 \mid d(a, e_2), & \text{se } \text{null}(e_1) \end{cases} \\
d(a, e_1 \mid e_2) &= d(a, e_1) \mid d(a, e_2) \\
d(a, e^*) &= d(a, e) e^*
\end{aligned}$$

Como exemplo, vamos derivar a linguagem $(a|\epsilon)bc$ em relação ao símbolo b .

$$\begin{aligned}
d(b, (a \mid \epsilon) b c) &\equiv \\
d(b, (a \mid \epsilon)) b c \mid d(b, b c) &\equiv \\
(d(b, a) \mid d(b, \epsilon)) b c \mid \epsilon d(b, b c) &\equiv \\
(\emptyset \mid \emptyset) b c \mid d(b, b c) &\equiv \\
\emptyset \mid c &\equiv c
\end{aligned}$$

Pode-se determinar se uma dada palavra formada pelos símbolos $a_1 \dots a_n$ é gerada pela expressão regular e aplicando sucessivamente a derivada e testando se a expressão resultante é anulável, ou seja, $a_1 \dots a_n \in L(e)$ se e somente se a expressão regular $d(a_n, d(a_{n-1}, \dots d(a_1, e)) \dots)$ for anulável.

2.5 Derivadas para Linguagens Livres de Contexto

O conceito de derivadas pode ser estendido para linguagens livres de contexto, adicionando-se os casos necessários para a derivação de símbolos não terminais que estão presentes nessas gramáticas. Uma GLC é formada por $\langle \Sigma, V, R, P \rangle$.

Essa mudança afeta tanto a definição de anulável quanto a definição de derivadas. Intuitivamente, a nova definição de anulável para GLC deve receber uma forma sentencial e retornar T se essa forma puder derivar ϵ . Vamos usar letras gregas do início do alfabeto (α e β) para denotar formas sentenciais da gramática e a letra v para denotar um não terminal. Logo, a nova definição de anuláveis seria $\text{null}_g : \{\Sigma \cup V\}^* \rightarrow \{T, F\}$:

$$\begin{aligned}
null_g(\epsilon) &= T \\
null_g(a) &= F \\
null_g(\alpha \beta) &= null_g(\alpha) \wedge null_g(\beta) \\
null_g(\alpha \mid \beta) &= null_g(\alpha) \vee null_g(\beta) \\
null_g(v) &= null_g(\alpha), \text{ se } v \rightarrow \alpha \in R
\end{aligned}$$

Subsequentemente, a definição de derivada se torna:

$$\begin{aligned}
d_g(a, \epsilon) &= \emptyset \\
d_g(a, a) &= \epsilon \\
d_g(a, b) &= \emptyset, \text{ se } a \neq b \\
d_g(a, \alpha \beta) &= \begin{cases} d_g(a, \alpha) \beta, & \text{se } \neg null_g(\alpha) \\ d_g(a, \alpha) \beta \mid d_g(a, \beta), & \text{se } null_g(\alpha) \end{cases} \\
d_g(a, \alpha \mid \beta) &= d_g(a, \alpha) \mid d_g(a, \beta) \\
d_g(a, v) &= d_g(a, \alpha) \text{ se } v \rightarrow \alpha \in R
\end{aligned}$$

Essas definições são matematicamente corretas, mas, como observado por Might ([MIGHT; DARAI; SPIEWAK, 2011](#)), computacionalmente elas podem levar à recursão infinita caso a gramática possua recursão à esquerda. A implementação proposta por Might tem como objetivo o uso de derivadas para parsing e emprega o uso de linguagem funcional com suporte a avaliação lazy e memoização para contornar esses problemas. Além desses problemas, Might também precisou implementar simplificações após cada passo da derivada para evitar o crescimento rápido da gramática e a degradação do tempo de parsing.

Uma segunda abordagem para derivadas, proposta por Henriksen ([HENRIKSEN; BILARDI; PINGALI, 2019](#)), descreve um método com custo quadrático em espaço e cúbico em tempo para a realização de parsing. A abordagem consiste em construir uma nova GLC a partir da gramática original. Além disso, cada não terminal é anotado com a sequência de símbolos sobre os quais foi derivado até um dado momento. Essa abordagem permite que se apliquem derivadas a gramáticas com recursão à esquerda. Uma segunda otimização é a eliminação de regras desnecessárias, que são inatingíveis a partir do símbolo de partida.

Para formalizar a abordagem de Henriksen, definimos a construção de uma gramática derivada $G_a = \langle \Sigma, V', R', S_a \rangle$ a partir da gramática original G , em relação a um

símbolo de entrada $a \in \Sigma$. O novo conjunto de não terminais V' passa a conter símbolos anotados com o histórico de derivação (como v_a).

A geração das novas regras baseia-se em uma função de derivação simbólica $\mathcal{D}_a$, que mapeia o lado direito das regras originais para o lado direito das novas regras. Em estilo declarativo, para cada regra na gramática original, geramos uma regra correspondente na gramática derivada:

$$\frac{v \rightarrow \alpha \in R}{v_a \rightarrow \mathcal{D}_a(\alpha) \in R'}$$

Onde $\mathcal{D}_a$ é definida por casos, dependendo se o primeiro símbolo da forma sentencial é um terminal ($b \in \Sigma$) ou um não terminal ($u \in V$):

$$\begin{aligned} \mathcal{D}_a(\epsilon) &= \emptyset \\ \mathcal{D}_a(b\beta) &= \begin{cases} \beta, & \text{se } a = b \\ \emptyset, & \text{se } a \neq b \end{cases} \\ \mathcal{D}_a(u\beta) &= \begin{cases} u_a\beta \mid \mathcal{D}_a(\beta), & \text{se } \text{null}_g(u) \\ u_a\beta, & \text{se } \neg \text{null}_g(u) \end{cases} \\ \mathcal{D}_a(\alpha \mid \beta) &= \mathcal{D}_a(\alpha) \mid \mathcal{D}_a(\beta) \end{aligned}$$

A grande vantagem dessas regras está no tratamento de não terminais (os casos envolvendo $u\beta$). Em vez de expandir o não terminal u recursivamente — o que causaria o loop infinito em gramáticas com recursão à esquerda —, o método introduz o novo não terminal u_a (que simboliza " u derivado por a "). Isso posterga a avaliação e transfere a resolução da recursão para a própria estrutura da nova gramática, transformando o problema de parsing em sucessivas reescritas da GLC.

Exemplo de Derivação Simbólica

Considere uma gramática muito simples com recursão à esquerda para expressões matemáticas:

$$\begin{aligned} S &\rightarrow S+T \mid T \\ T &\rightarrow x \end{aligned}$$

Vamos calcular a gramática derivada para o símbolo de entrada x . Sabendo que nem S nem T são anuláveis ($\neg \text{null}_g(S)$ e $\neg \text{null}_g(T)$), aplicamos a função $\mathcal{D}_x$ às regras originais para obter os novos não terminais S_x e T_x :

1. **Regra $S \rightarrow S + T$:** Como S é não terminal e não anulável, geramos $S_x \rightarrow \mathcal{D}_x(S + T) \Rightarrow S_x \rightarrow S_x + T$.
2. **Regra $S \rightarrow T$:** Como T é não terminal e não anulável, geramos $S_x \rightarrow \mathcal{D}_x(T) \Rightarrow S_x \rightarrow T_x$.
3. **Regra $T \rightarrow x$:** Como x é um terminal e coincide com o símbolo de entrada, geramos $T_x \rightarrow \mathcal{D}_x(x) \Rightarrow T_x \rightarrow \epsilon$.

A gramática derivada resultante (focando nos estados atingíveis a partir do novo símbolo inicial S_x) é finita e bem definida:

$$S_x \rightarrow S_x + T \mid T_x$$

$$T_x \rightarrow \epsilon$$

$$T \rightarrow x \quad (\text{mantida pois é referenciada em } S_x)$$

Este exemplo ilustra como a abordagem de Henriksen contorna a recursividade à esquerda ao gerar o símbolo S_x de forma simbólica, evitando a expansão contínua que ocorreria nas derivadas estritas.

3 Desenvolvimento

A proposta da ferramenta é permitir que o aluno explore o funcionamento de um APN, com clareza a respeito do seu não determinismo. Adicionalmente, queremos que o aluno seja capaz de testar a corretude de um autômato. Este último passo requer uma especificação da linguagem que o APD deve aceitar. Para tanto, empregamos GLCs como uma forma de especificar a linguagem a ser implementada.

O processo de correção automatizado consiste na geração de palavras que devem ser aceitas pelo APD. Para esta etapa, empregamos o uso de um método baseado em derivadas livres de contexto.

Este capítulo apresenta como a ferramenta foi desenvolvida e descreve a arquitetura de software utilizada, assim como as bibliotecas e técnicas de programação. Ele está organizado da seguinte forma: a seção 3.1 descreve a implementação da ferramenta; a subseção 3.1.1 descreve de forma geral como o APD e sua execução foram modelados; por fim, a seção 3.1.2 apresenta a implementação do gerador de palavras a partir de uma dada GLC.

3.1 Implementação

A ferramenta foi desenvolvida utilizando a linguagem de programação JavaScript, com foco na orientação a objetos, com o auxílio de HTML5 e CSS para a sua apresentação visual. A aplicação ocorre completamente no *front-end* e não possui partes executadas em servidor. Também não foi utilizado nenhum tipo de *framework* para auxiliar na organização do código. Além das tecnologias nativas, foi utilizada a biblioteca JavaScript Cytoscape.

A biblioteca Cytoscape é utilizada para a manipulação de desenhos em HTML5, com foco na visualização de grafos. Ela foi escolhida por ser totalmente *open source* e porque grafos são a forma mais comum de representação de autômatos, utilizada, por exemplo, no livro didático (VIEIRA, 2006). A biblioteca foi empregada apenas para o desenho dos grafos, tendo toda a parte de simulação de autômatos sido desenvolvida exclusivamente durante este trabalho. O código-fonte está disponível no GitHub, e o software encontra-se hospedado de forma gratuita por meio do GitHub Pages para testes.

O sistema desenvolvido originalmente possuía os seis tipos de máquinas de estados estudados na disciplina de Fundamentos Teóricos da Computação na UFOP. Porém, devido ao tempo limitado e ao escopo abrangente, foi tomada a decisão de seguir apenas com a representação de APNs e a correção de exercícios. Caso haja interesse, essa versão beta pode ser encontrada [neste repositório](#), ao passo que o código mais recente está disponível

[neste repositório](#).

3.1.1 Modelagem do APD e sua execução

Ao desenvolver este projeto, foi feita uma divisão entre a representação lógica e a representação gráfica do APN. A representação lógica do APN e a sua execução dentro do sistema acontecem por meio de quatro classes principais: **Transition**, **APN**, **Snapshot** e **APNRunner**.

Cada transição foi representada utilizando objetos da classe **Transition**, que possui quatro atributos simples: **sym**, que é o caractere a ser lido na palavra; **stkR**, que é o valor que deverá ser desempilhado da pilha; **stkW**, que é o valor que deverá ser empilhado na pilha; e o atributo **state**, que é o estado de destino daquela transição.

A classe **APN** ficou responsável por reunir os estados e as transições, além de alguns métodos auxiliares. Cada estado é representado por um número simples, e o conjunto de estados, por um *array*. Já o conjunto de transições foi representado por um mapa que tem como valor um *array* de objetos da classe **Transition** e como chave o número referente ao estado de origem da transição.

A classe **APN** possui o método **delta**, que utiliza os atributos do seu mapa de transições para determinar quais transições podem ser atingidas a partir de um estado *s*, um caractere de palavra *chr* e um topo de pilha *r*, retornando esse conjunto de transições na forma de um *array*. Esse método percorre cada transição que tem sua chave no mapa igual a *s* (já que sua chave no mapa é seu estado de origem) e, em cada uma, verifica se o atributo **sym** é igual a *chr* ou à string vazia, e se o atributo **stkR** é igual a *r* ou à string vazia. Se ambas as condições forem satisfeitas simultaneamente, essa transição é considerada válida e é adicionada ao *array* que será retornado ao final do método.

O maior desafio ao representar APNs neste projeto foi modelar o seu não determinismo, dado que existem muitas execuções simultâneas do mesmo autômato. A forma adotada para representar cada uma das execuções simultâneas causadas pelo não determinismo foi a criação da classe **Snapshot**, na qual cada **Snapshot** representa um ponto da execução de um dos possíveis caminhos da máquina, podendo vários **Snapshots** existir simultaneamente.

Para modelar isso, a classe **Snapshot** conta com os seguintes atributos: **state**, que representa o estado em que a execução se encontra; **pos**, que representa a posição do caractere lido na palavra que está sendo processada; e **stack**, que representa o conteúdo da pilha naquele momento da execução.

Para a visualização do usuário, é criado um grafo de *snapshots* construído na classe **Graph**. Essa classe pode ser utilizada para a criação de grafos de qualquer tipo de objeto JavaScript, sendo composta por dois mapas: um que tem como chave um ID e como valor

um único objeto que representa um nó; e um segundo mapa, que representa as arestas entre os nós utilizando os valores de ID do primeiro mapa, no qual a chave é o nó de origem e o valor é o nó de destino da aresta.

A classe **APNRunner** é a principal responsável pela execução do autômato. Ela possui o atributo **apn**, que é um objeto da classe **APN**; **input**, que é a palavra que será executada; **snapshots**, que é um *array* de objetos da classe **Snapshot**; e **graph**, que é um objeto da classe **Graph**.

O principal método responsável pela execução do autômato é o **step**. Ele percorre cada um dos *snapshots* atuais utilizando busca em largura (mantendo um *array* com todas as instâncias mais recentes) e verifica as transições válidas a partir deles por meio do método **delta** da classe **APN**. As transições válidas resultantes são utilizadas para criar novos *snapshots*, que passam a ser os atuais, finalizando um **step**. Adicionalmente, a cada **step** concluído, os novos *snapshots* são adicionados ao grafo, criando-se uma aresta entre cada *snapshot* original e os gerados a partir dele.

A simulação acontece por meio de sucessivas chamadas à função **step**, até que todos os *snapshots* atuais já tenham concluído a leitura da palavra ou até que se chegue a um limite de *loops* de execução definido pelo usuário na interface gráfica.

3.1.2 Gerando palavras a partir de uma GLC

O processo para geração de palavras consiste no uso de derivadas para sintetizar uma palavra a partir de uma GLC. Essa abordagem é simples e tem a vantagem de garantir, por definição, que a palavra sintetizada deverá necessariamente ser aceita por qualquer APN que implemente a respectiva GLC. A técnica consiste em, iterativamente, derivar a GLC em relação a um símbolo proveniente do conjunto *First* do símbolo de partida. A cada passo, inspecionamos se o símbolo de partida é anulável; em caso afirmativo, a concatenação de todos os símbolos derivados até o momento forma uma palavra de entrada válida que deve ser aceita pelo APN.

Intuitivamente, o conjunto *First*, frequentemente empregado em reconhecedores sintáticos LL(1) (AHO et al., 2006), pode ser entendido como o conjunto de todos os símbolos terminais que podem ser produzidos imediatamente por uma forma sentencial. Por exemplo, uma regra na forma $A \rightarrow aAb \mid c$ possui como conjunto *First* os elementos $\{a, c\}$. Dada uma gramática $G = \langle \Sigma, V, R, P \rangle$, seja $F(A)$ o conjunto *First* do não terminal A . Inicialmente, para cada $A \in V$, $F(A) = \emptyset$. Formalmente, o conjunto $First(X)$ pode ser definido pela aplicação das seguintes regras, repetidas até que nenhum dos conjuntos sofra alterações:

1. Se X é um terminal, então $First(X) = \{X\}$.

2. Se X é um não terminal com regra $X \rightarrow Y_1 Y_2 \dots Y_k$ (para algum $k \geq 1$), e se existe algum j tal que $\neg \text{nullable}(Y_j)$ com $0 \leq j \leq k$, então $\text{First}(X) = \text{First}(X) \cup \text{First}(Y_i)$ para $0 \leq i \leq j$.

O conjunto de não terminais anuláveis (*nullables*) de uma GLC é o conjunto de todos os não terminais que podem gerar a palavra vazia (ϵ). Esse conjunto pode ser computado pela aplicação das seguintes regras até que o conjunto pare de mudar:

1. Para toda produção na forma $A \rightarrow \epsilon$, temos que $A \in \text{nullables}$.
2. Para toda produção na forma $A \rightarrow B_1 \dots B_n$, se $B_1 \dots B_n \in \text{nullables}$, então $A \in \text{nullables}$.

A implementação desses conceitos em JavaScript se beneficia do uso do tipo de dados **Set** para a implementação de *nullables*, e de **Map** para a implementação do *First*. Esses conceitos foram codificados em uma classe que modela uma GLC, formada por um **Map** que armazena as produções da gramática. A API desenvolvida permite a adição de regras e a definição do símbolo de partida, além de disponibilizar o método **derivate**, que cria uma cópia da gramática contendo as regras derivadas.

A implementação do método da derivada também copia a tabela *First* da gramática original. Como o *First* é computado por um algoritmo de ponto fixo e todas as produções da gramática base são copiadas para a derivada, o novo conjunto *First* não precisa ser computado do zero, baseando-se no conjunto original. Isso permite que apenas adicionemos à tabela as novas produções mais relevantes. A API da gramática contém ainda o método **prune**, responsável por remover produções inatingíveis a partir do símbolo de partida.

Finalmente, o processo para a geração de uma palavra pode ser descrito pela seguinte sequência de passos, dada a gramática $G = \langle \Sigma, V, R, P \rangle$:

1. Inicialize a palavra w com ϵ .
2. Inicialize o conjunto $WRDS$ com $\emptyset$.
3. Se P pode gerar ϵ , faça $WRDS = WRDS \cup \{w\}$.
4. Compute o $\text{First}(P)$.
5. Escolha, aleatoriamente, um símbolo a de $\text{First}(P)$.
6. Compute a derivada de G em relação a a .
7. Remova as produções inatingíveis de G .
8. Atualize a lista de produções anuláveis.
9. Retorne ao passo 3.

3.2 Uso do software

A interface do software pode ser dividida em duas partes principais: criação/edição de APNs e execução do APN com base em uma palavra. A parte de criação permite: instanciar estados; editar o nome do estado; defini-lo como final, inicial, ambos ou nenhum; excluí-lo; criar transições; editar valores da transição; excluir transições; baixar o APN desenvolvido; importar um APN desenvolvido anteriormente; mover livremente os elementos; e aplicar *layouts* prontos do Cytoscape para organizar a visualização.

Já a parte de execução do APN permite: definir a palavra de entrada; estipular o número limite de *loops* para a função **step**; escolher a condição de aceitação entre FS (estado final e pilha vazia), S (pilha vazia) ou F (estado final); iniciar e interromper a simulação; avançar ou retroceder um **step**; reiniciar a execução; e visualizar os dados completos de cada *snapshot* individualmente ao clicar nele.

Para exemplificar a utilização, vamos ao cenário de um autômato simples. Para criar cada estado na ferramenta, basta clicar no painel esquerdo do sistema, conforme a Figura 1a. Clicando em um estado com o botão direito, abrem-se as opções de edição: "inicial" e "final" (que alternam o estado daquele nó), além de "renomear" e "excluir". Essas funções são ilustradas na Figura 1b.

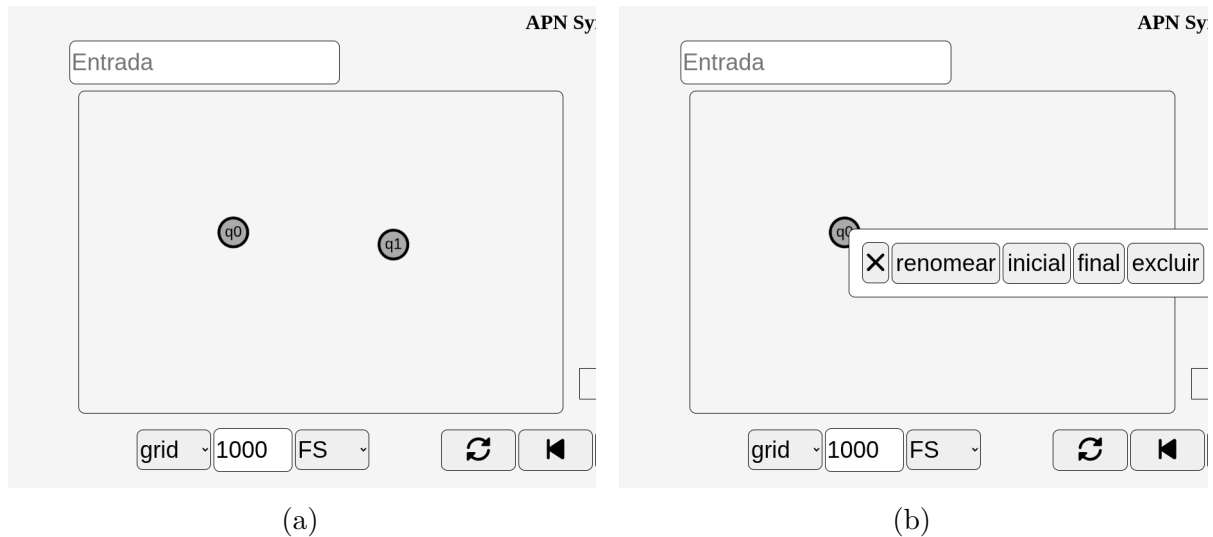


Figura 1 – a Criação do estado, b Editando o estado

Para criar uma transição, clique no estado de origem com o botão esquerdo e, em seguida, no estado de destino. A transição gerada tem, por padrão, o valor vazio em todos os elementos, como mostra a Figura 2a. Ao clicar em uma transição com o botão direito, suas opções de edição são exibidas (Figura 2b): "excluir" e "renomear", que permitem alterar o caractere de leitura, o caractere a ser desempilhado e o caractere a ser empilhado. Um exemplo de autômato simples construído na ferramenta pode ser visto na Figura 3.

Ao inserir a palavra de leitura no campo "Entrada" (como a palavra "0101", mostrada

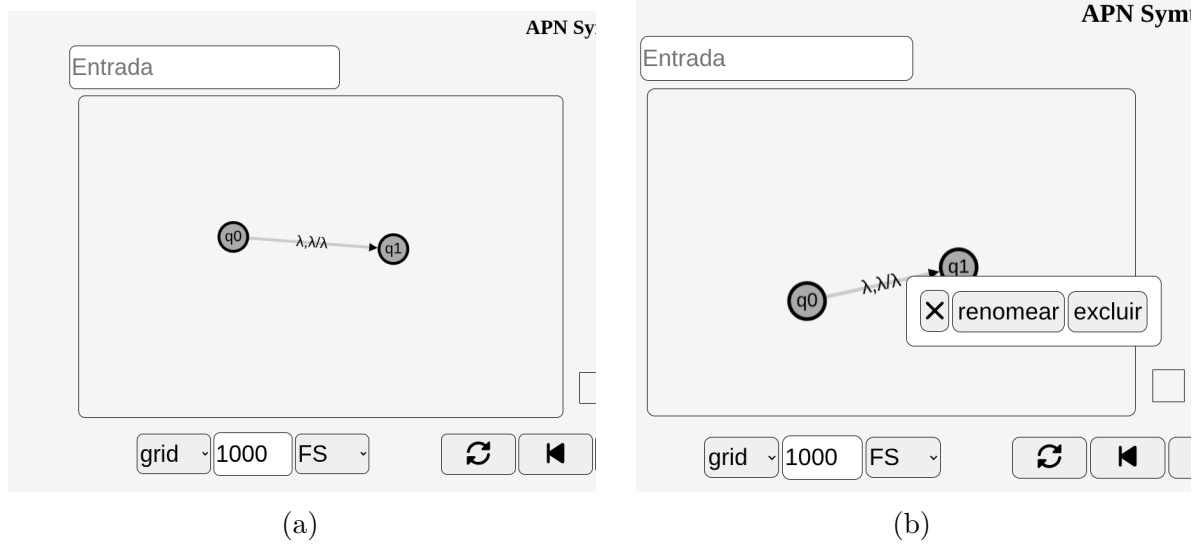


Figura 2 – a Criação da transição, b Editando a transição

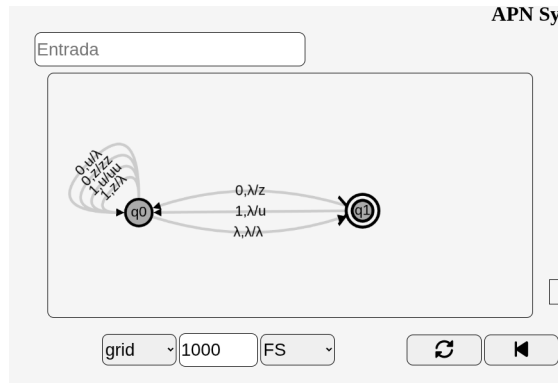


Figura 3 – Um APN simples

na Figura 4) e clicar no botão inferior com ícone de *play*, o sistema calcula a execução completa do autômato. Nesse momento, o botão de *play* transforma-se no ícone de *stop*, permitindo interromper a demonstração se desejado.

Durante a execução, o campo da palavra é substituído por uma tabela detalhando cada caractere e seu respectivo índice, destacando o último símbolo lido para facilitar a compreensão (Figura 5a). Além disso, no painel direito é possível visualizar as etapas de execução (os *snapshots*). Aqueles associados ao caractere recém-lido são destacados em verde (Figura 5b). Em cada nó do painel é possível ver: o estado atual na máquina, o índice do caractere lido na palavra e o topo da pilha (até três elementos).

No painel direito, as arestas entre os *snapshots* desenhadas em azul indicam os caminhos que podem levar a um estado de aceitação (Figura 6a), enquanto as cinzas representam ramos sem sucesso. Ao clicar em um *snapshot*, é possível ver sua pilha completa no centro da tela e visualizar o estado em questão destacado no desenho principal (Figura 6b). Os botões de avançar e retroceder permitem iterar passo a passo pelo processamento, atualizando os nós em verde (Figura 6c).

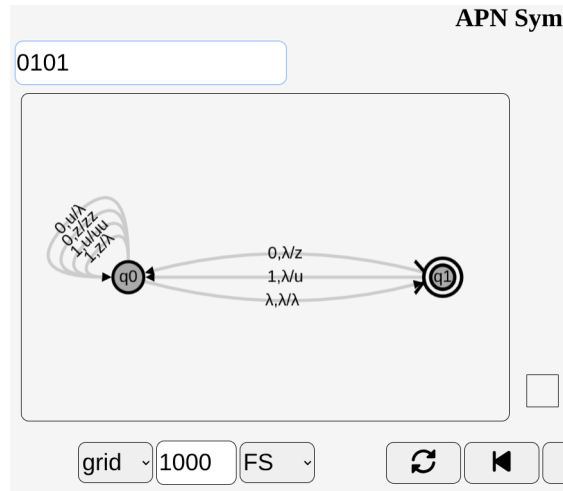
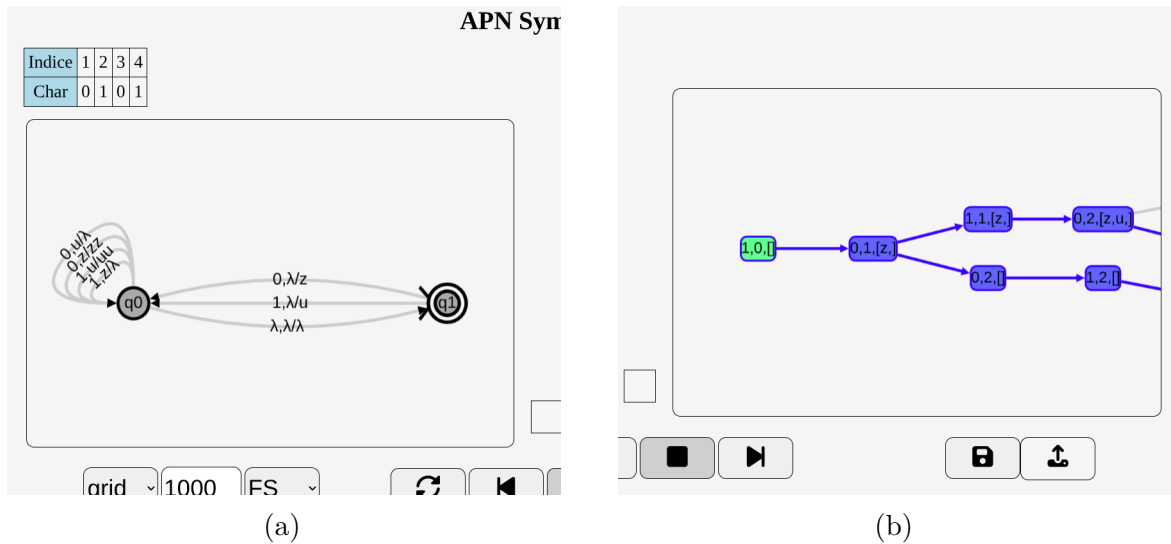


Figura 4 – Entrada da palavra 0101

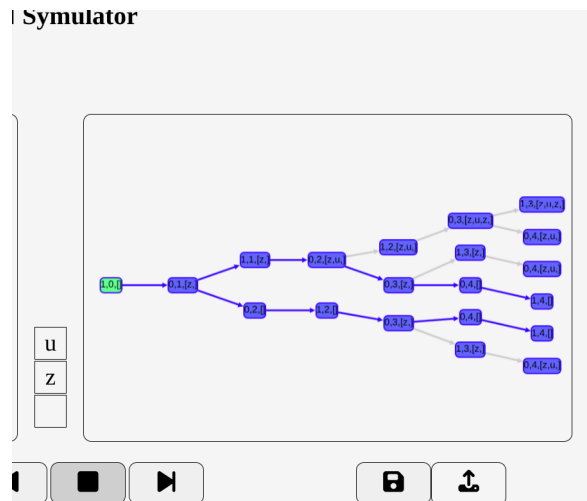


(a)

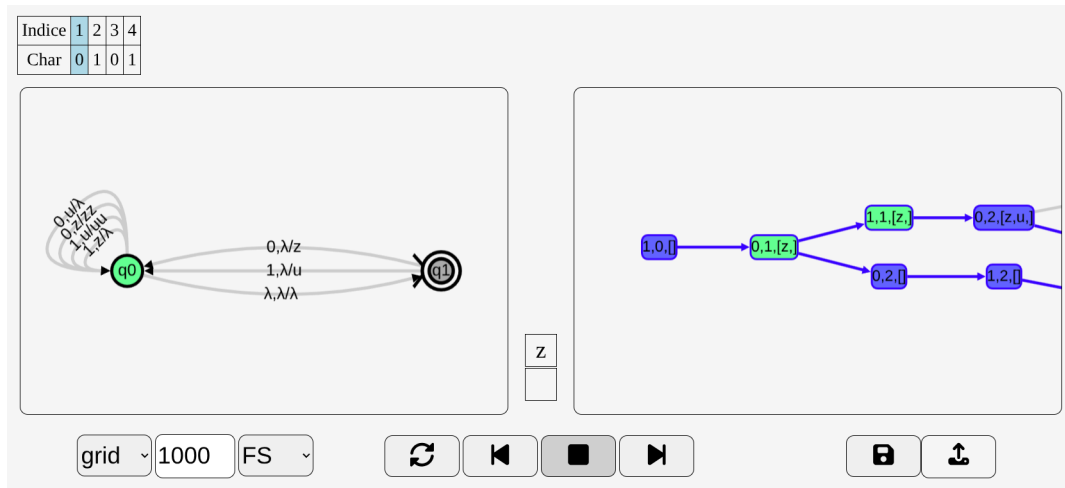
(b)

Figura 5 – a Tabela de caracteres, b Visualização dos *snapshots*

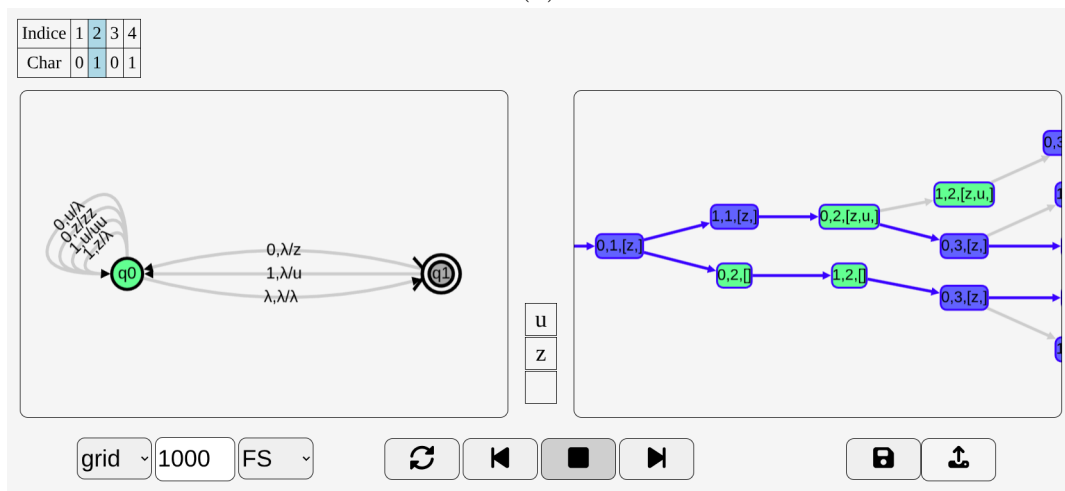
No canto inferior esquerdo, encontram-se três botões de configuração (Figura 7a): o primeiro ajusta o *layout* visual; o segundo define o limite numérico de *loops* para interromper execuções infinitas; e o terceiro seleciona o critério de aceitação (FS, S ou F). Por fim, no canto inferior direito, há dois botões (Figura 7b): o ícone de disquete realiza o *download* do autômato criado em formato JSON, e o ícone de seta faz o *upload* de um JSON contendo uma máquina previamente salva.



(a)



(b)



(c)

Figura 6 – a Caminho de aceitação, b Clique em *snapshot*, c Botão de avançar

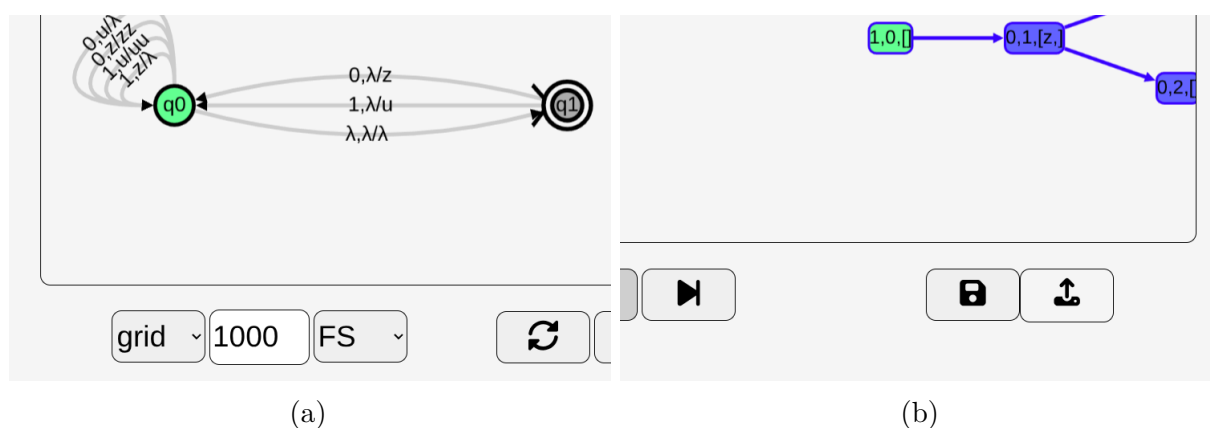


Figura 7 – **a** Botões de configuração, **b** Botões de *download/upload*

4 Resultados

Este capítulo descreve os resultados finais do desenvolvimento da ferramenta proposta, abordando o que foi feito, as limitações da ferramenta desenvolvida, assim como possíveis trabalhos futuros e expansões do projeto.

Foi desenvolvido um construtor gráfico de APNs baseado em grafos que podem ser moldados pelo usuário, além de um painel que ilustra a execução de uma determinada palavra no APN e dos vários caminhos possíveis tomados pelo não determinismo. Esse painel também marca zero ou mais caminhos que resultem em aceitação. Ao executar uma determinada palavra no autômato, o usuário pode escolher entre os três tipos de aceitação: pilha vazia (S), estado final (F) ou ambos (FS). O desenvolvimento da interface foi realizado pensando em um uso mais rápido e menos verboso, em detrimento de opções mais detalhadas, focando principalmente em precisar de um menor número de cliques na tela para executar cada ação.

Os APNs construídos dentro da ferramenta podem ser baixados em formato JSON e podem ser importados de arquivos do computador do usuário para continuar com estudos anteriores. O sistema foi criado para fins gráficos e didáticos, portanto é focado em pequenos autômatos de pilha e pode apresentar lentidão dependendo do número de não determinismos e do tamanho da palavra analisada.

O sistema não foi projetado para teste em massa de autômatos grandes, mas pode ter sua eficiência refinada para melhores desempenhos em casos mais exigentes. Também não consegue evitar loops infinitos dentro do autômato, por isso existe um modificador de máximo de loops de iteração que pode ser alterado pelo usuário, pois impedir totalmente loops infinitos em APNs ainda não tem uma resolução efetiva ou não foi encontrada durante a revisão para este projeto. Navegadores modernos, no momento, utilizam apenas uma thread para sua interface gráfica ou frontend, e o projeto não utiliza backend, uma vez que uma das principais ideias era manter esse sistema hospedado de forma gratuita e estável, e um backend dificultaria essa meta. Por esse motivo, o projeto todo é executado em uma única thread, mesmo para calcular os não determinismos.

O desenvolvimento do software para outras máquinas de estado que não APNs chegou a ser iniciado, porém, devido ao tempo reduzido, foi feita a escolha de reduzir o escopo do projeto. Em trabalhos futuros, é possível finalizar a implementação das demais máquinas de estados. Além disso, não foi realizado um teste com alunos para saber sobre a real efetividade ou eficiência da ferramenta no ensino didático, portanto, esse seria um dos principais tópicos abordados em trabalhos futuros.

5 Conclusão

Este trabalho teve o objetivo de desenvolver uma ferramenta para auxiliar os alunos dos cursos da área de computação no aprendizado de linguagens formais e máquinas de estados, focando principalmente em autômatos de pilha não determinísticos (APNs), dado que esses conhecimentos são fundamentais, porém complexos e abstratos.

Entre as principais contribuições do projeto desenvolvido estão: a criação de APNs; a visualização dos múltiplos caminhos de execução de um APN passo a passo; correção automática de exercícios; tudo em uma plataforma completamente online, sem necessidade de instalação no computador.

Apesar disso, o sistema possui algumas limitações, sendo a execução de autômatos que podem criar um número exponencial de múltiplos caminhos quando computados com palavras grandes demais. Essa limitação varia muito de acordo com o hardware do usuário, já que é executado completamente em frontend, porém isso teoricamente não chega a ser um grande problema, já que o sistema foi desenvolvido pensando em executar autômatos voltados a exemplos didáticos e não em simulações de grande porte para pesquisas.

Dessa forma, conclui-se que a ferramenta desenvolvida demonstra potencial de auxiliar os alunos no melhor entendimento acerca de autômatos de pilha não determinísticos, porém não foi realizada pesquisa com os alunos para determinar sua eficiência ou efetividade, sendo esse um dos principais tópicos a serem abordados em trabalhos futuros, juntamente com a implementação de outras máquinas de estados além dos APNs e, talvez, o desenvolvimento de um backend para o sistema, que poderia melhorar a performance do sistema e permitir a execução de autômatos maiores.

Referências

- AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2nd. ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2006. ISBN 0321486811. Citado na página 32.
- BRZOZOWSKI, J. A. Derivatives of regular expressions. *J. ACM*, Association for Computing Machinery, New York, NY, USA, v. 11, n. 4, p. 481–494, out. 1964. ISSN 0004-5411. Disponível em: <<https://doi.org/10.1145/321239.321249>>. Citado na página 24.
- BURCH, C. *Autosim*. 2008. <<https://cburch.com/proj/autosim/index.html>>. Acessado em: 24 jun. 2025. Citado na página 18.
- DICKERSON, K. *Automaton Simulator*. 2021. <<https://automatonsimulator.com/>>. Acessado em: 24 jun. 2025. Citado na página 18.
- DOTY, D. *UC Davis Automaton Simulator*. 2020. <<https://web.cs.ucdavis.edu/~doty/automata/>>. Acessado em: 24 jun. 2025. Citado na página 18.
- Free Software Foundation. *grep(1) — Linux User's Manual*. [S.l.], 2026. Disponível offline via comando 'man grep'. Disponível em: <<https://man7.org/linux/man-pages/man1/grep.1.html>>. Citado na página 21.
- HENRIKSEN, I.; BILARDI, G.; PINGALI, K. Derivative grammars: a symbolic approach to parsing with derivatives. *Proc. ACM Program. Lang.*, Association for Computing Machinery, New York, NY, USA, v. 3, n. OOPSLA, out. 2019. Disponível em: <<https://doi.org/10.1145/3360553>>. Citado na página 27.
- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd. ed. Boston, MA, USA: Addison-Wesley, 2006. ISBN 0-321-46225-4. Citado na página 21.
- JUKEMURA, A. S.; NASCIMENTO, H. A. D. do; UCHÔA, J. Q. Gam - um simulador para auxiliar o ensino de linguagens formais e de autômatos. *25º Congersso da Sociedade Brasileira de Computação*, 2005. Acessado em: 24 jun. 2025. Disponível em: <https://www.academia.edu/download/69969649/GAM-Um_Simulador_para_Auxiliar_o_Ensino_20210920-880-j6ritm.pdf>. Citado na página 17.
- KIENETZ, E. B.; CANAL, A. P. Simulador de autômatos finitos determinísticos - automograph. *Disc. Scientia. Série: Ciências Naturais e Tecnológicas*, v. 5, n. 1, p. 1–9, 2004. ISSN 1519-0625. Trabalho de Iniciação Científica - PROBIC. Acessado em: 24 jun. 2025. Disponível em: <<https://periodicos.ufn.edu.br/index.php/disciplinarumNT/article/download/1176/1113>>. Citado na página 17.
- MIGHT, M.; DARAIS, D.; SPIEWAK, D. Parsing with derivatives: a functional pearl. In: *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming*. New York, NY, USA: Association for Computing Machinery, 2011. (ICFP '11), p. 189–195. ISBN 9781450308656. Disponível em: <<https://doi.org/10.1145/2034773.2034801>>. Citado na página 27.

RENSSELAER; UNIVERSITY, D. *JFLAP*. 2018. <<https://www.jflap.org/>>. Acessado em: 24 jun. 2025. Citado na página 18.

VIEIRA, L. F. M.; VIEIRA, M. A. M.; VIEIRA, N. J. Language emulator, uma ferramenta de auxílio no ensino de teoria da computação. *Anais do Congresso*, 2003. Departamento de Ciência da Computação, UFMG. Acessado em: 24 jun. 2025. Disponível em: <<http://homepages.dcc.ufmg.br/~mmvieira/publications/Wei-languageEmulator.pdf>>. Citado na página 17.

VIEIRA, N. *Introdução aos fundamentos da computação: linguagens e máquinas*. Pioneira Thomson Learning, 2006. ISBN 9788522105083. Disponível em: <<https://books.google.com.br/books?id=62ieMQAACAAJ>>. Citado 2 vezes nas páginas 21 e 30.

WHITE, T. M.; WAY, T. P. jfast: a java finite automata simulator. *SIGCSE Bull.*, Association for Computing Machinery, New York, NY, USA, v. 38, n. 1, p. 384–388, mar. 2006. ISSN 0097-8418. Disponível em: <<https://doi.org/10.1145/1124706.1121460>>. Citado na página 18.

ZUZAK, I.; JANKOVIC, V. *FSM Simulator*. 2025. <https://ivanzuzak.info/noam/webapps/fsm_simulator/>. Acessado em: 24 jun. 2025. Citado na página 17.